

In [16]:

```
import os
import ssl
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader

# Disable SSL verification to bypass local network proxy/firewall restrictions
ssl._create_default_https_context = ssl._create_unverified_context

# Hardware Acceleration Setup
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device for training: {device}")

# Execution Hyperparameters
BATCH_SIZE = 32
LEARNING_RATE = 0.001
NUM_EPOCHS = 3
DATA_DIR = './data'
```

Using device for training: cpu

In [11]:

```
# Image Normalization transform pipeline
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Download and instantiate training and test splits
train_dataset = torchvision.datasets.MNIST(root=DATA_DIR, train=True, download=True,
test_dataset = torchvision.datasets.MNIST(root=DATA_DIR, train=False, download=True,

# Optimized Data Loaders
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_wo
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False, num_wor
```

In [18]:

```
class LabCNN(nn.Module):
    def __init__(self):
        super(LabCNN, self).__init__()
        # Input Tensor Shape: [Batch, 1, 28, 28]
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, paddin
        self.conv2 = nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3, paddi
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

        # Dense Fully Connected Topology (Size after dual downsampling is 7x7)
        self.fc1 = nn.Linear(in_features=64 * 7 * 7, out_features=512)
        self.fc2 = nn.Linear(in_features=512, out_features=10)

    def forward(self, x):
        # Feature Extraction Layer Pipeline
        x = self.pool(F.relu(self.conv1(x))) # Dimensions: [Batch, 32, 14, 14]
        x = self.pool(F.relu(self.conv2(x))) # Dimensions: [Batch, 64, 7, 7]

        # Multidimensional Matrix Flattening Sequence
        x = x.view(-1, 64 * 7 * 7)

        # Classification Projection Layers
```

```

        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

# Instantiate network on target runtime device
model = LabCNN().to(device)
print(model)

```

```

LabCNN(
  (conv1): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=3136, out_features=512, bias=True)
  (fc2): Linear(in_features=512, out_features=10, bias=True)
)

```

```

In [19]: criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)
print("Loss function and optimizer initialization complete.")

```

Loss function and optimizer initialization complete.

```

In [20]: print("Starting model training process...")

for epoch in range(NUM_EPOCHS):
    model.train()
    running_loss = 0.0

    for inputs, labels in train_loader:
        inputs, labels = inputs.to(device), labels.to(device)

        # Gradient Vector Reset Phase
        optimizer.zero_grad()

        # Forward Pass and Error Backward Propagation
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    print(f"Epoch [{epoch + 1}/{NUM_EPOCHS}] completed. Average Loss: {running_loss}")
print("Training phase finished.")

```

Starting model training process...
Epoch [1/3] completed. Average Loss: 0.1111
Epoch [2/3] completed. Average Loss: 0.0391
Epoch [3/3] completed. Average Loss: 0.0248
Training phase finished.

```

In [21]: print("Running final model evaluation on test dataset...")
model.eval()
correct = 0
total = 0

with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs.data, 1)

```

```
total += labels.size(0)
correct += (predicted == labels).sum().item()

print(f"Final Test Accuracy: {(100 * correct / total):.2f}%")
```

Running final model evaluation on test dataset...
Final Test Accuracy: 99.00%

In []: